# Phantom Dependencies in AI-Generated Build Manifests for Cyber-Physical Systems: Measurement and Horizon-Anchored Admission

Anonymous Author(s)

***Abstract*—Code-generating language models write build manifests for cyber-physical systems (CPS) and declare dependencies that do not exist. Whoever registers such a name compromises the next build, and the common check that every dependency resolves cannot stop it. We anchor dependency evidence to the producer's knowledge horizon, the date at which the model's weights demonstrably existed, and gate admission on three-state evidence. We prove that a name first registered after the horizon is never admitted, for any weights, threshold and gating time, given valid record dates. For blocking, the gate equals a record-age filter with an explicit unknown state, which asks for evidence where registry dates are missing or one-sided. Over 8 CPS ecosystems, 96 tasks and 13 open-weight models of 1.5–32.8B parameters, 64.0% of the answers that declared registry packages named a nonexistent one (60.2% of those that finished), and resampled answers repeated a phantom of the greedy answer in 33.2% of cases. If every phantom open to any account holder were registered, a resolves-now check would admit 46% of the manifests with a phantom at once and the gate none; a replay of 9 squats on real installers and of a stand-in in an ESP-IDF build agrees.**

***Index Terms*—software supply chain, package hallucination, slopsquatting, cyber-physical systems, build integrity, admission control, LLM-generated code**

## I. Introduction

The software of a cyber-physical system (CPS) is assembled from third-party packages. A flight controller pulls sensor drivers and protocol stacks from the Arduino Library Manager or the PlatformIO registry, an ESP32 node pulls components from the ESP Component Registry, a robot pulls message and navigation packages through the ROS 2 distribution and `rosdep`, and the Linux gateway beside a programmable logic controller pulls Modbus, OPC UA and CAN libraries from PyPI or Node-RED nodes from npm. Whatever enters this build is signed and shipped over the air to devices that move things. Attacks on open-source supply chains are well catalogued [1]–[3], and the XZ Utils backdoor showed how far an adversary will go to place code in a dependency that many systems trust [4].

Code-generating LLMs add a new supplier to this chain: assistants and coding agents write manifests, run install commands and invent package names. Spracklen et al. found that 19.7% of 2.23 million package references produced by 16 models were hallucinated, 21.7% for open-source and 5.2% for commercial models [5]; a 2026 re-evaluation of five frontier models still measured 4.62–6.10% [6]. Many hallucinated names recur across runs [5], which makes them worth registering: an empty package under one such name drew more than 30,000 downloads in three months [7]. The attack, known among practitioners as *slopsquatting*, joins typosquatting and dependency confusion in the family of package-confusion attacks [8], [9].

Two gaps motivate this paper. The first is *coverage*. Published measurements target general-purpose registries: PyPI and npm [5], [7], and crates.io [10], [11]. The registries of embedded and robotic software differ in ways that matter for this threat: the Arduino Library Manager registers a new library automatically once its checks pass [12], PlatformIO resolves owner-less names that its own documentation calls a source of conflicts [13], the ESP Component Registry maps names without a namespace to the vendor namespace [14], and new ROS packages enter the distribution through pull requests checked against formal criteria [15]. Whether code models hallucinate in these ecosystems, and whether an outsider could exploit the result, is unknown.

The second gap is the *defense*. The common control checks that every dependency resolves. This check is not monotone in time: the attack makes the name resolve, so a check that blocked yesterday's phantom admits today's squat, and age rules ("block packages younger than $N$ days") only delay the admission. Checking generated names against a list of packages registered before each model's training cut-off has been proposed for PyPI, npm and crates.io [10], but not implemented or evaluated; it relies on the model's stated cut-off, which cannot be checked and can differ from what the weights encode [16], and is silent on records and registries without dates. The installers' date filters hide files or versions published after a cut-off [17], [18], while a squat is a new registry record, and two questions stay open: which cut-off to use for a change written by a model, and what to do where the date it needs is missing or one-sided. Most CPS registries expose no dates to their installers, and PyPI reports only the upload times of files that still exist, which bound the creation of a record from above only. A two-valued filter must treat such packages as unavailable.

Consider a manifest from our study. Asked for a PlatformIO project, Qwen2.5-Coder 1.5B declared 2 dependencies, one of which, `ph-52af75b0d8` (a pseudonym, as for every name an outsider may be able to register), is not in the registry. Today the installation fails. If an adversary registers the name, a resolves-now check admits the manifest from that day on (ADMIT), and a 30-day age rule decides REJECT on the first day and ADMIT 30 days later. The question that does not

change with time is whether the registry record existed when the model's weights did.

We answer the first question with the producer. Every producer of a change has a *knowledge horizon* $h$: not its true, unknowable training cut-off, but a verifiable *upper bound* on it — the date at which the exact weights we ran demonstrably existed on the model hub (at or after the real cut-off). A registry record first created after $h$ cannot be what the producer knew, whatever the producer knew about the project behind it. We answer the second with evidence semantics. We encode establishment as a three-state evidence item in the supply-chain integrity index (SCII), an evidence-graph index that separates absent from refuted evidence [19], and gate admission with a non-compensable rule. Where record dates exist, a squatted name is refuted; where they are missing or one-sided, the gate asks for evidence, such as the history of the registry repository, instead of guessing. For the manifest above it decides REJECT at any gating time, because the squatted record would postdate the model's horizon (2024-09-18). Our contributions are:

- **Evidence model.** Dependency evidence for AI-originated changes with four items (resolution, establishment at a *verifiable* weight-existence horizon rather than a stated cut-off, content pinning, publisher provenance) in three states, bounds on registry *records* rather than projects, and four properties: non-compensability, horizon soundness, monotone confirmation, and the insufficiency of the graded index alone. The cut-off-bounded list is due to [10]; new are the verifiable horizon, the third state that turns a missing or one-sided date into a request, record dates recovered from the histories of CPS registries, and their evaluation (Sections IV and VI).
- **Measurement.** PHANTOMDEPS-CPS, 96 tasks over 8 CPS ecosystems with registry snapshots, and a study with 13 models: phantom prevalence by ecosystem, model, task tier and answer quality (well-formed, malformed, truncated, checked against the ecosystems' own parsers), recurrence, which phantoms outsiders can register, and an audit of post-horizon matches (Sections V–VI).
- **Gate evaluation.** Nine policies, among them two- and three-valued date filters at the horizon, on the generated manifests in the observed state and in simulated squat worlds; what registry histories decide; how threshold, weights and change size trade blocking against availability; lock files and provenance from the real tools; and a replay of squats against real installers and the ESP-IDF build system.

## II. Background and Related Work

*Supply-chain attacks and package confusion.* Ladisa et al. organise 107 attack vectors on open-source supply chains into a taxonomy [1]; earlier reviews collected malicious packages [2] and mapped weak links such as few-maintainer dominance and expired e-mail domains [20], [21]. Package confusion generalises typosquatting [8], and dependency confusion exploits the precedence between private and public registries [9]. Further work covers vulnerability propagation [22] and practices after SolarWinds, Log4j and XZ Utils [23], [24].

TABLE I
CLOSEST WORK ON LLM PACKAGE HALLUCINATION. *Anchor*: THE TIME BOUND A DEFENSE WOULD USE. *No-date*: HANDLES RECORDS/REGISTRIES WITHOUT DATES. *Impl.*: THE TIME-ANCHORED DEFENSE IS IMPLEMENTED AND EVALUATED (NOT A DIFFERENT DEFENSE SUCH AS DECODING OR RAG). *Inst.*: EXERCISES REAL INSTALLERS.

| Work | Registries / anchor | No-date | Impl. | Inst. |
|---|---|---|---|---|
| Spracklen [5] | PyPI, npm / fixed date | – | n/a | – |
| Krishna [10] | +crates / stated cut-off | – | proposed | – |
| HFUZZER [25] | PyPI / cut-off (labels) | – | n/a | – |
| PackMonitor [26] | PyPI / none | – | n/a† | – |
| Zheng [11] | crates / none | – | n/a | – |
| **This paper** | +5 CPS reg. / verif. horizon | ✓ | ✓ | ✓ |

†PackMonitor constrains decoding to a current list (a different, non-temporal defense); a name already registered passes.

*LLM-generated code and package hallucination.* Package hallucination was measured for Python and JavaScript [5], further languages [7], [10], Rust crates [11], by fuzzing [25], and on a 2026 cohort [6]. For the cut-off-bounded list of Krishna et al. [10, §4.6] (Section I), the effective cut-off can differ from the stated one [16], which is why we bound the horizon by when the weights demonstrably existed. PackMonitor restricts install commands at decoding time to a current, undated list [26], which an adversary changes by registering a name. To our knowledge no prior work measures package hallucination in the registries of embedded and robotic software (Arduino, PlatformIO, ESP-IDF, ROS 2, MicroPython), bounds admission by a verifiable horizon, or decides a change whose dates are missing or one-sided; Table I compares the closest work.

*Integrity and admission mechanisms.* TUF [27] and its automotive profile Uptane [28] protect the update channel; in-toto [29] and SLSA [30] attest the steps and provenance of a build; Sigstore lowers the cost of signing [31]. Package managers offer date-anchored resolution: npm installs only versions available before a date or older than a number of days [17], and uv ignores distributions uploaded after a cut-off and treats those without an upload time as unavailable [18]. The indices that the Arduino, ROS and MicroPython installers read carry no dates at all. Process frameworks (NIST SP 800-161r1 [32], SSDF [33], IEC 62443-4-1 [34]) and the EU Cyber Resilience Act [35] require dependency control but do not say how to decide on an individual change. The evidence-graph index that we extend [19] distinguishes absent from refuted evidence and combines a graded index with a non-compensable gate. The installers filter by calendar date or package age, not by a bound tied to the producer of the change, and treat a package without a date as unavailable.

## III. Threat Model and Problem

*Pipeline.* A producer $P$ (developer, LLM assistant or coding agent) emits a change $\Delta$ to a CPS project at generation time $t_g$. The change declares a set $D(\Delta)$ of dependencies, each a name in an ecosystem $e$ with registry $\mathcal{R}_e$. Before the build

is signed and released, a gate decides at gating time $t_b \geq t_g$ among ADMIT; REJECT, for a squat whose registry record postdates the model; REQUEST, when the pipeline must first obtain missing evidence, such as the date of a library in a registry that serves none; and REVIEW, when every mandatory check passes but a human must decide, as for an established dependency without a pinned digest. Fig. 1 shows the setting.

*Knowledge horizon.* For a closed-book model, the latest time of information that can ground $P$'s suggestions is its training cut-off $c$. Reported cut-offs are not always accurate [16], and the creation date of a model repository bounds nothing, because a repository can be created before any weights are uploaded. We therefore take as the *horizon* $h(P)$ the date at which the weights we ran demonstrably existed: the commit on the model hub that last changed the vendor's weight files from which our file was converted, or, if those files changed later, the commit that added our file itself, identified by its SHA-256. Weights cannot reflect data published after they existed, so $c \leq h$. A late bound shifts records created in $(c, h-\delta]$ from refuted to established. A squat informed by this model can be registered only after the model was released, hence after $h$; the window matters for squats prepared without the model, for example from the phantoms of other models (RQ2 measures how many recur across models). A retrieval-augmented agent has a later horizon that must be supported by a logged lookup; we leave such agents to future work.

*Adversary.* A *squat* is the registration of a phantom name by the adversary at a time $t_a$. The adversary observes or predicts phantom names (for instance by querying the same models) and can register any free name in an open registry, at any time after $h$, without provenance. It cannot publish into a namespace it does not control, cannot alter the times that a registry records, and does not compromise the registry or the build infrastructure. Out of scope are squats registered before $h$: by the time the producer writes they are established records, and the horizon cannot tell them from legitimate ones; provenance evidence and registry-side defenses have to cover them. Also out of scope are takeover of established packages (the XZ Utils pattern), malicious logic in packages that existed before $h$, and re-registration of deleted names (Section VIII). Our squat worlds (Section V-E) register every phantom that nobody owns, or every phantom that is open for registration, and so bound the adversary from above.

*Three dates.* A package has a *project* (source repository, documentation), a *record* in a registry under a name, and *artifacts* (versions, files) published into that record. Installers fetch artifacts of the record that the declared name resolves to, so the gate reasons about records: a project older than its record does not make the record older, and the record of one name says nothing about another record of the same project.

*Definition 1 (Phantom and post-horizon match):* A registry declaration $d$ (a manifest entry or install command naming a registry package) is a *phantom* at time $t$ if its name, as the installer resolves it, does not resolve in $\mathcal{R}_e$ at $t$, is not provided by the toolchain (board core, framework, language) and is not declared by the change itself. A declaration that resolves to a record first created after $h(P)$ is a *post-horizon match*: the producer may have known the project, not this record.

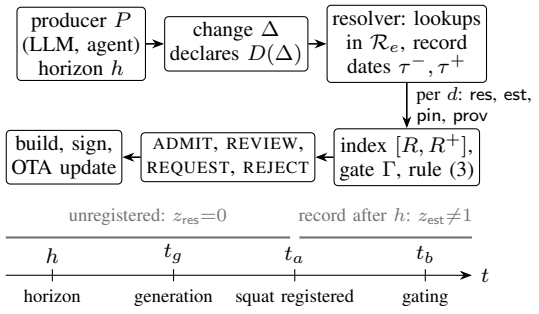



Fig. 1. Gating AI-originated dependency declarations. Bottom: whatever the gating time, a phantom is never admitted: before the squat it does not resolve; after it, its establishment is refuted where the registry dates record creation, and undecided, held for evidence, where it bounds it from above only.

*Requirements.* The gate must (S1) never admit a phantom or a post-horizon match under any tuning; (S2) make evidence gaps explicit instead of silently admitting or rejecting; and (S3) run in time linear in the change size, plus registry lookups.

## IV. Horizon-Anchored Evidence for Dependencies

### A. Evidence for one dependency

For each dependency $d \in D(\Delta)$ that a change declares, the gate collects four evidence items, each in one of three states: confirmed (1), refuted (0) or absent ($\perp$). Absent means that no evidence was obtained (not produced, not reachable, or not decidable); refuted means that evidence contradicts the requirement. The first two items are mandatory:

- res (resolution): 1 if $d$ resolves in $\mathcal{R}_e$ at gating time or is provided by the toolchain; 0 if the registry is authoritative for the name and it does not resolve; $\perp$ if resolution cannot be decided (a VCS URL, a path, a legacy numeric identifier, a failed lookup, a package outside the snapshots). A name resolves only as the installer resolves it: PyPI names after its normalisation, PlatformIO and ESP names up to case as their APIs match them, all other names exactly (Cargo, for one, rejects a crate written with _ for - or in another case); a PyPI project without files and a crate whose versions are all yanked do not resolve, because the installer finds nothing to install. A registry match takes precedence over the toolchain list.
- est (establishment): with a lower bound $\tau^-$ and an upper bound $\tau^+$ on the first publication time $\tau(d)$, horizon $h$ and margin $\delta \geq 0$,

$$z_{\text{est}} = \begin{cases} 1 & \tau^+ \leq h - \delta, \\ 0 & \tau^- > h, \\ \perp & \text{otherwise (including unknown bounds).} \end{cases} \tag{1}$$

- pin (content pinning): 1 if the pipeline records a digest of the resolved artifact in a lock file (or the declaration pins a full commit hash), 0 on a digest mismatch, $\perp$ otherwise.

- prov (publisher provenance): 1 if provenance is present and valid, 0 if verification fails, $\perp$ if absent.

The margin $\delta$ keeps records created just before the horizon undecided: a model can hardly have learned a package from the last weeks of its training data. The bounds come from the registry and describe the record. Where the registry dates the creation of the record (PlatformIO `added_at`, ESP `created_at`, the crates.io `pubtime` of the first version, npm `time.created`), both bounds equal that time. PyPI reports only the upload times of files that still exist: the earliest bounds the record from above and is no lower bound, because earlier files may have been deleted. The Arduino Library Manager and the ROS distribution keep their registries in git, and their indices carry no dates; we date a library or a package by the first change on the main line of the registry repository that added it (a pull-request merge that GitHub creates, or a maintainer's commit), so the date is set by the registry and not by the submitter. Arduino moves are linked where the log shows them, and libraries of the initial import get an upper bound only. ROS packages are dated in the distribution file of the release the tasks name (Humble). MicroPython packages get an upper bound from the first commit to their manifest in micropython-lib by a maintainer; commits by other committers carry dates a contributor chose. An owner-less PlatformIO name may install any same-name package, so its bounds span all candidates and stay undecided if one candidate has no date. A pipeline that looked a name up and found it unregistered holds a further lower bound, its *absence record*, for any record that appears later. Toolchain-provided names inherit the evidence of the platform and are confirmed.

### B. From a manifest to a decision

The gate $\Gamma$ of a change is 0 if some item of some declaration is refuted, $\perp$ if none is but a mandatory item is absent, and 1 otherwise. Once the gate passes, a graded index decides between ADMIT and REVIEW: it encodes the optional evidence a project demands before a change goes in unreviewed. With item weights $\alpha_s \geq 0$ that sum to 1 and a threshold $\theta \in (0, 1]$, the lower and upper labels of a declaration count absent items as failed and as confirmed, respectively,

$$\ell(d) = \sum_s \alpha_s [z_s(d) = 1], \qquad \ell^+(d) = \sum_s \alpha_s [z_s(d) \neq 0], \tag{2}$$

and the index of the change is $R = \prod_d \ell(d)$, with $R^+ = \prod_d \ell^+(d)$; any completion of the absent items yields an index in $[R, R^+]$. The decision is

$$\mathrm{dec}(\Delta) = \begin{cases} \text{REJECT} & \Gamma = 0 \text{ or } (\Gamma = 1 \wedge R^+ < \theta), \\ \text{REQUEST} & \Gamma = \perp, \\ \text{ADMIT} & \Gamma = 1 \wedge R \geq \theta, \\ \text{REVIEW} & \text{otherwise.} \end{cases} \tag{3}$$

The gate is non-compensable: no weight can offset a refutation, and a missing mandatory item is reported instead of being scored. Rejects and requests come from the gate, reviews from the index. With this gate, $\Gamma = 1$ implies that no item is refuted, hence $R^+ = 1 \geq \theta$; the $R^+$ clause matters only for the index-only variant below, and we keep the rule in the form of [19].

*General form.* The index is the supply-chain integrity index (SCII) of [19] for one release step; we restate what this paper uses and prove the properties it relies on, so nothing below depends on that work. A profile $\mathcal{P} = (V, E, w, S, S^*, \alpha, \theta)$ fixes a directed acyclic graph of process vertices whose root $\rho$ is the release, edge weights $w_{uv} \in [0, 1]$ on edges $u \to v$ from a supplying to a consuming vertex, and for each vertex $v$ the items $S_v$ it expects, with weights summing to 1, and the mandatory subset $S_v^* \subseteq S_v$. Labels (2) are computed per vertex, so $\ell(d)$ is the label of the vertex $v_d$ of declaration $d$, with $\ell(v) = \ell^+(v) = 1$ for a vertex without items, and propagate with a product kernel,

$$r(v) = \ell(v) \prod_{(u,v) \in E} \big(1 - w_{uv}\,(1 - r(u))\big), \tag{4}$$

giving $R = r(\rho)$ and, with $\ell^+$ for $\ell$, $R^+$. A change instantiates the graph with one vertex $v_d$ per declaration and an edge of weight 1 into the dependency-resolution vertex, because a compromised dependency compromises the build; that vertex carries no evidence of its own and is the root of the part of the graph that changes, so (4) reduces to the products above.

### C. Properties

*Proposition 1 (Non-compensability):* If $z_{\mathsf{res}}(d) = 0$ for some $d \in D(\Delta)$, then $\mathrm{dec}(\Delta) = \text{REJECT}$ for every $\alpha$ and $\theta$.

*Proof:* A refuted item sets $\Gamma = 0$, and (3) rejects whenever $\Gamma = 0$. ∎

*Proposition 2 (Horizon soundness):* Let $P$ declare $d$, where $d$ is neither provided by the toolchain nor declared in $\Delta$, and no package of that name, as the installer resolves it, was registered in $\mathcal{R}_e$ at any time $t \leq h(P)$. Assume that every upper bound the resolver reports is valid within the margin, $\tau^+ \geq \tau(d) - \delta$, where $\tau(d)$ is the first publication of the package the installer would fetch. Then, with res and est mandatory, for every gating time $t_b$, every registry state at $t_b$, and every $\alpha, \theta$, $\mathrm{dec}(\Delta) \neq \text{ADMIT}$.

*Proof:* If $d$ does not resolve at $t_b$, then $z_{\mathsf{res}} = 0$, or $z_{\mathsf{res}} = \perp$ if the lookup fails. Otherwise the package it resolves to was first published at $\tau(d) > h$, so $\tau^+ \geq \tau(d) - \delta > h - \delta$, or $\tau^+$ is unknown, and (1) gives $z_{\mathsf{est}} \neq 1$. In every case a mandatory item is refuted or absent, so $\Gamma \in \{0, \perp\}$ and the decision is REJECT or REQUEST. ∎

The two baselines behave differently. A resolves-now check admits $d$ from the moment of registration; an age rule with threshold $N$ admits it $N$ days later. Proposition 2 holds at every $t_b$ because $h$ does not move, and it covers a post-horizon match, whatever the reason the model wrote its name. Its guarantee starts at the horizon the gate uses: an upper bound on the true cut-off leaves registrations between the two unprotected (Section III), a further reason to keep the bound tight. The margin also absorbs small offsets between the date a registry records and the time it serves a package, such as the delay between a merge and the next index build. Correct

lower bounds matter only for refutations; an invalid $\tau^-$ can turn REQUEST into REJECT, never into ADMIT.

*Scope.* The guarantee is exactly this: a name whose record is first created after $h$ is never admitted, when record dates are valid within $\delta$ and the registry is not compromised. It says nothing about a package that already existed at $h$ (a classic typosquat or a malicious old package), a new *version* of such a package (content pinning and provenance, the optional items, bear on that), or a compromised publisher. It ranges over the declarations the model wrote, not over the transitive closure the installer resolves later: the horizon says nothing about packages the model never named, which other evidence, such as pins in the lock file, has to cover.

*Proposition 3 (Monotone confirmation):* Replacing an absent item by a confirmed one never decreases $R$ or $R^+$, and never turns ADMIT into another decision.

*Proof:* By (2), $\ell$ grows by $\alpha_s \geq 0$ and $\ell^+$ is unchanged. The kernel in (4), and the product it reduces to, is non-decreasing in every $r(u)$ and in $\ell(v)$, so $R$ does not decrease and $R^+$ is unchanged. ADMIT requires $\Gamma = 1$ and $R \geq \theta$; confirming an item cannot create a refutation or an absent mandatory item, so both conditions persist. ∎

*Proposition 4 (Insufficiency of the index alone):* The index-only rule ("REJECT if $R^+ < \theta$, ADMIT if $R \geq \theta$, else REVIEW") admits a squatted dependency for some admissible $\alpha, \theta$.

*Proof:* Take $\alpha = (0.5, 0.05, 0.4, 0.05)$ for $(\mathsf{res}, \mathsf{est}, \mathsf{pin}, \mathsf{prov})$ and $\theta = 0.85$. A squatted name resolves ($z_{\mathsf{res}} = 1$), the pipeline pins whatever it resolved ($z_{\mathsf{pin}} = 1$), $z_{\mathsf{est}} = 0$ and $z_{\mathsf{prov}} = \perp$. Then $\ell = 0.9 \geq \theta$ and the index-only rule admits it, while (3) rejects because $\Gamma = 0$. ∎

Exactly: a squat has $z_{\mathsf{est}} \neq 1$ and $z_{\mathsf{prov}} = \perp$, so its label is $\alpha_{\mathsf{res}} + \alpha_{\mathsf{pin}}$ if pinned and $\alpha_{\mathsf{res}}$ if not, and the index-only rule can admit a manifest containing it only if $\alpha_{\mathsf{est}} + \alpha_{\mathsf{prov}} \leq 1 - \theta$, or $\alpha_{\mathsf{res}} \geq \theta$ without a pin; Section VI-C samples how often profiles satisfy this. A rule that rejects whenever $z_{\mathsf{est}} \neq 1$ would also be safe, but it rejects every change whose record dates are missing or one-sided.

*Relation to date filters.* Consider a filter on the record-creation time at $h$: it admits a record known to predate $h$ and hides a later one. Give it a third answer, *unknown*, for a record it cannot place, and let it ask for evidence then: this filter ($\mathrm{P3}_\perp$ in Section V-E) is exactly the gate. With $\alpha_{\mathsf{pin}} = \alpha_{\mathsf{prov}} = 0$, $\delta = 0$ and no optional item refuted ($\mathrm{P3}_\perp$ ignores pins and provenance), rule (3) decides every change as $\mathrm{P3}_\perp$ does, since $\Gamma = 1$ then implies $R = 1$; the analysis checks this on every manifest and world, and a unit test on random dependency sets. The comparison is to this record-age filter, not to the installers' upload- or version-time filters (Section V-E). The graded index adds what blocking does not need: optional evidence (pinning, provenance) that decides between ADMIT and REVIEW, and the margin $\delta$.

*Cost.* The index and gate take one pass over the change, $O(|D(\Delta)|)$ here and $O(|V|+|E|)$ in general; registry lookups dominate and are cached.

### D. Implementation

PHANTOMDEPS-CPS (Python) renders prompts, queries OpenAI-compatible endpoints, classifies declarations against snapshots and cached registry answers, evaluates the policies and writes every number of this paper with a provenance tag. Unit tests check Propositions 1–4 on random profiles and through the pipeline, the equivalence with $\mathrm{P3}_\perp$ on 4,000 random dependency sets, registry-history dating and the extraction rules. The analysis aborts if the rule admits a squat, if the classification came from other code or snapshots, or if the data contradict one of the statements of Section VI that it checks, such as that every policy but P0 rejects every manifest with a phantom.

## V. STUDY DESIGN

We ask four research questions. **RQ1** (prevalence): how often do code models declare phantom dependencies when they produce build manifests for CPS ecosystems, and how does this vary by ecosystem, model, task tier, prompt family and answer quality? **RQ2** (exposure): do phantoms recur across models and runs, do suggested names already resolve to records created after the model's horizon, and which phantoms could an outsider register? **RQ3** (gating): how do baseline policies, date filters and the horizon-anchored rule decide on the generated manifests, in the observed registry state and when phantoms are squatted, and what do they cost in availability? **RQ4** (evidence and cost): how often can establishment be decided, what lock files and provenance do the tools supply, and what does gating cost? The protocol was fixed before the study run; its change log lists every later change and why it was made.

### A. Ecosystems and snapshots

Table II lists the eight ecosystems: the firmware layer (Arduino, PlatformIO, ESP-IDF, embedded Rust, MicroPython), robotics (ROS 2) and the gateway layer of industrial systems (PyPI, Node-RED nodes on npm). Static indices (Arduino and MicroPython indices, the `humble` and `jazzy` distribution files, the `rosdep` databases) were downloaded and hashed, and the registry repositories of Arduino, ROS and MicroPython cloned at recorded commits; other registries were queried per name, with every answer cached with its retrieval time (latest: 2026-10-28). Toolchain names come from nine Arduino cores, the ESP-IDF components (v5.5 and master), the MicroPython documentation and the Python standard library, matched up to case and separators (which can only lower phantom counts).

### B. Task corpus

The corpus has 96 tasks, 12 per ecosystem (SHA-256 prefix `d11355380088`). Each task names a board or host and a function in one of eleven categories (sensing, actuation, IoT, fieldbus, positioning, security, control, data, display, wireless, robotics), but no package, so the dependency names are chosen by the model. In ROS, two tasks name a standard message type and thus its package, and a few name the framework that

TABLE II
ECOSYSTEMS: HOW A NAME ENTERS THE REGISTRY, SOURCE OF THE RECORD-DATE BOUNDS, DIGEST LOCKING, RESOLVED DECLARATIONS WITH A DECIDABLE est, AND DECISIONS OF RULE (3) ON CLEAN MANIFESTS (SECTION V-E) PER TASK ECOSYSTEM (A ADMIT, ? REVIEW OR REQUEST, X REJECT).

| Ecosystem | New names | Dates | Lock | est decided | $C$: A/?/X |
|---|---|---|---|---|---|
| Arduino LM | open, automated | merge log | no | 357/359 | 0/95/0 |
| PlatformIO | open, owner ns. | API | no | 338/393 | 0/80/0 |
| ESP-IDF | open, namespace | API | yes | 35/37 | 2/14/0 |
| ROS 2 | PR review | merge log | no | 1795/1796 | 0/68/0 |
| crates.io | open | index | yes | 1439/1440 | 1/108/0 |
| MicroPython | PR review | git (upper) | no | 93/93 | 14/47/0 |
| PyPI | open | files (upper) | yes | 657/680 | 36/165/0 |
| npm | open, name rules | API | yes | 345/345 | 40/67/0 |

defines the interface. Half the tasks of each ecosystem went to a *common* tier (well-documented devices and protocols, such as a BME280 sensor or MQTT) and half to a *niche* tier (less common fieldbuses such as J1939, DNP3 or IEC 61850, secure-update functions, specific GNSS or IMU parts) before any generation. Two prompt families wrap each task: F1 asks for the manifest and the source code, F2 for the manifest or install commands only. For example, the F2 prompt of task `platformio-07` reads: *"Create a PlatformIO project for an ESP32-S3 DevKitC using the Arduino framework. Task: Poll holding registers of a PLC over Modbus TCP and forward the values to MQTT. Provide only the complete platformio.ini, including lib_deps. Do not write the source code."* All prompts share one system message (in the artifact).

### C. Models and generation

The study ran 13 open-weight models from 7 vendors, 1.5 to 32.8 billion parameters (Table III): Qwen2.5-Coder [36], Llama 3.1 and 3.2 [37], DeepSeek-Coder [38] and its V2 [39], Granite 3.3 [40], Gemma 3 [41], Phi-4 [42], and the reasoning models Qwen3 [43] and gpt-oss [44]. Each model is the hub GGUF file we pinned by its SHA-256, quantised as the Ollama library's default, and was served by Ollama 0.40.11 with the library's chat template and default parameters (our system message replaces a library default), one request at a time, with an 8,192-token context on a laptop GPU with 12 GB (the two largest models partly in system memory). Each prompt of both families received one greedy answer (seed 20240628) under one budget of 1536 tokens for both families; the reasoning models got 4096 tokens, and only their final answer is analysed. Every (model, prompt) pair whose greedy answer contained a phantom was asked 5 more times at temperature 0.7 to measure persistence, and every cut answer was generated again with 4096/7680 tokens as a sensitivity analysis. A pilot with smaller models on a CPU host had validated the harness beforehand; the artifact holds it. Primary metrics use the prompt families answered by every model (F1, F2), so that every model contributes the same prompts. The horizon of each model is the date at which its weights demonstrably existed (Section III), read from the commit history of the model hub; it lies within 251 hours of the creation of each repository, and the files we ran, identified by SHA-256, were committed 0.1 to 345.6 days later.

TABLE III
MODELS (PRIMARY FAMILIES): HORIZON $h$, TOKEN BUDGET, ANSWERS CUT BY IT, AND YIELD (ANSWERS WITH AN EXECUTABLE DECLARATION); TABLE IV GIVES THEIR PHANTOM RATES.

| Model | Horizon $h$ | Budget | Cut | Yield |
|---|---|---|---|---|
| Qwen2.5-C 1.5B | 2024-09-18 | 1536 | 85 | 167/192 |
| Llama-3.2 3B | 2024-09-18 | 1536 | 60 | 134/192 |
| Qwen2.5-C 7B | 2024-09-17 | 1536 | 24 | 183/192 |
| DS-Coder 6.7B | 2023-10-29 | 1536 | 15 | 168/192 |
| Llama-3.1 8B | 2024-07-18 | 1536 | 51 | 152/192 |
| Qwen3 8B | 2025-04-28 | 4096 | 34 | 153/192 |
| Granite-3.3 8B | 2025-04-09 | 1536 | 11 | 152/192 |
| Gemma-3 12B | 2025-03-12 | 1536 | 107 | 166/192 |
| Qwen2.5-C 14B | 2024-11-06 | 1536 | 25 | 164/192 |
| Phi-4 14B | 2024-12-11 | 1536 | 9 | 164/192 |
| DS-Coder-V2 16B | 2024-06-14 | 1536 | 21 | 188/192 |
| gpt-oss 20B | 2025-08-04 | 4096 | 102 | 107/192 |
| Qwen2.5-C 32B | 2024-11-10 | 1536 | 18 | 185/192 |

### D. Extraction and classification

Only executable declarations are extracted: manifest entries (`lib_deps`, `idf_component.yml`, `package.xml`, `Cargo.toml`, requirement lists, `package.json`, `library.properties`) and install commands (`arduino-cli`, `pio`, `idf.py`, `cargo add`, `mip`, `pip`, `npm`, `apt` for ROS packages) in code context, with arguments ending at the first token that reads as prose. ESP-IDF manifests that list dependencies instead of mapping them are extracted and flagged, since a developer would fix the syntax and keep the names. Prose recommendations are ignored, so the counts are lower bounds on what a reader would try to install. The ecosystem of a declaration follows from its syntax. A declaration is classified as *exists*, *phantom*, *toolchain*, *local* (a ROS package of the same workspace), *non-registry* (URL or path), *legacy identifier*, *special* (not a package, such as the ESP-IDF version constraint), *unsupported* (a package outside the snapshots, such as one of another ROS distribution; gated with undecided resolution) or *unresolved* (lookup failure).

To separate answer quality from hallucination, every manifest file is parsed independently of the extractor with the parser of its ecosystem's tooling (`tomllib`, JSON, `packaging`, `configparser`, `catkin_pkg`, the component manager's YAML schema); answers count as well-formed, malformed, commands only or cut by the token limit, and on well-formed files the extractor's names are compared with the parsers'.

### E. Gate evaluation

A manifest is the set of non-local, non-special declarations in one response. It is *unsafe* ($U$) if it contains a phantom, *post-horizon* ($H$) if it contains a post-horizon match but no phantom, and *clean* ($C$) otherwise. *Clean* means only that our lookups found no phantom and no post-horizon match; such a manifest need not build (it may name a version that does not exist) and is not certified safe. We compare nine policies: P0

admits everything; P1 (resolves now) rejects a manifest with a declaration known not to resolve and lets URLs, paths and failed lookups pass, as an installation would; P2 adds an age rule: every package with a known date must be at least 30 days old; P3 counts a registry package only if its registry *record* is known to have been created by $h$, treats a record without a date as unavailable, and does not date-filter URLs, paths and failed lookups; P3$_\perp$ is P3 with a third answer: it rejects what is known to be newer than $h$, admits what is known to be older, and requests evidence for everything else, URLs and paths included; I is the index-only rule; T (two-state) reads an absent mandatory item as refuted; O (optimistic) reads every absent item as confirmed; and SCII is rule (3). P3 is an abstract *record-age* policy: it tests what a squat changes, a new record under an old-looking name. The installers' filters test something else: uv `--exclude-newer` filters distribution *upload* times and npm `before` *version* publish times, which decide the version that installs (a 2020 package with a 2026 release passes P3 but loses that release under the filters). They coincide with P3 only when a name's earliest artifact dates its record, so we use them in the replay (Section VI-D) and P3 in the evaluation. The profile is assigned, not fitted: $\alpha = (0.4, 0.3, 0.2, 0.1)$, $\theta = 0.85$, $\delta = 30$ days. No publisher provenance is evaluated, so $\mathsf{prov} = \perp$ for every registry package.

We evaluate three worlds. In the *observed* world, registry state is the snapshot and $\mathsf{pin} = 1$ only where the lock file that the ecosystem's own tool wrote for the answer records a digest for the declaration (Cargo, npm and uv, Section VI-E); other declarations are unpinned. For comparison we also decide with the capability alone, $\mathsf{pin} = 1$ wherever the tooling can lock digests (Table II). In the *squat* world, every phantom is registered by the adversary one day after the snapshot, without provenance but pinned wherever the tooling can lock (the worst case for blocking), and the gate runs $L$ days later; registered names without installable files stay with their owners. The evidence of a squat follows the resolver of its ecosystem: a refuted $\mathsf{est}$ where the registry dates the creation of records, absent where it bounds them from above only (PyPI, MicroPython); the gate meets the name for the first time after the squat, and a variant gives it the absence record of our snapshot lookup. The *open squat* world registers only phantoms that any account holder could register. To see what registry histories contribute, we also decide every manifest without the dates derived from them (Arduino, ROS, MicroPython), as a filter that reads only the installers' indices would. The sweep draws 1000 profiles with $\alpha \sim \mathrm{Dirichlet}(1, 1, 1, 1)$ and $\theta \sim U(0.5, 0.95)$, and a grid over $\theta$ and $\alpha_{\mathsf{prov}}$ (the other weights keep their ratios) measures availability on clean manifests. Finally, we replay squats against real installers (pip, uv, npm, Cargo) and registries on the local host, and build an ESP-IDF application that declares a stand-in for one (Section VI-D).

### F. Statistics and ethics

Rates come with 95% percentile bootstrap intervals (2,000 replicates, seed 20240628) that resample tasks, because the answers of all models to one task, and the declarations within one answer, are not independent [45]; persistence resamples (model, prompt) pairs. Cells with fewer than ten registry declarations or from a single task are reported but not interpreted. Nothing was published to a public registry, no package code ran outside the replay (local registries, harmless packages), lookups were read-only and cached, and lock files were written in resolve-only modes. Names an outsider may be able to register appear as keyed pseudonyms and will be reported to the registry operators before publication; a name that is also a real package elsewhere stays readable there.

## VI. Results

The reported configuration produced 2496 responses in the primary families (562 hit the token limit). Of these, 2083 (83.5%) contained at least one executable declaration, for 12029 declarations in total: 5143 exist, 6049 are phantoms, 513 are toolchain names, and 324 fall in the other classes of Section V-D. Of the 413 responses without a declaration, 119 contained no code block at all. The answers contained 1306 manifest files, of which the ecosystems' own parsers rejected 362 (140 of 290 `package.xml`, 118 of 159 `idf_component.yml`, 77 of 298 `Cargo.toml`, 26 of 278 `platformio.ini`, 1 of 281 requirement lists); 578 registry declarations (561 of them phantoms) come from malformed ESP-IDF manifests. On the well-formed files, the extractor declared 5275 of the names the official parsers found, missed 0 and added 0 (precision 100.0%, recall 100.0% against the tools' own reading; both find requirement lists with the same rule, so recall on them is an upper bound). The check and three LLM pre-screens of 200 answers each (not counted as validation) found extraction errors that the extractor now handles (comments, unfenced requirement lists, MicroPython's `upip`, irregular ESP-IDF manifests); the manual validation uses a fresh sample. Two annotators from one university group labelled a stratified sample of 200 responses jointly; against their labels, extraction had a precision of 97.6% (CI [93.4, 99.9]) and a recall of 94.3% (CI [84.9, 99.5]). All 27 spurious and 5 missed items are dependencies by URL that the extractor names by package, the annotators by URL; 50 of the 65 missed items are tables of one `Cargo.toml` that does not parse. The errors cluster in few responses, so the intervals resample responses: extraction was exact on 191 of 200 (95.5%, Wilson CI [91.7, 97.6]), with a miss in 9 and a spurious item in 5. This step validates extraction, not the phantom/exists/toolchain classification, which rests on the registry lookups and name rules of Section V-D; an independent double annotation of both is future work.

### A. RQ1: prevalence

Of 11192 registry declarations, 6049 were phantoms (54.0%, 95% CI [47.0, 60.6]). At the response level, 1291 of 2018 responses with a registry declaration contained a

phantom (64.0%, CI [58.2, 69.5]). The pooled rate weights models by how much they declare: Llama-3.2 3B contributed 15% of the registry declarations, and the mean of the per-model rates is 48.1%. Declaration counts are sensitive to repetition loops: the largest single answer, cut by the limit, held 214 registry declarations, 212 of them phantoms (Llama-3.2 3B). Table IV breaks the rate down. Among the 8 ecosystems with at least ten registry declarations, the highest rate was in ESP-IDF (95.2%) and the lowest in PyPI (24.8%). Common-tier tasks yielded 44.5% (2408/5411, CI [35.9, 52.8]) and niche tasks 63.0% (3641/5781, CI [53.5, 72.7]). Answer quality matters, and the subsets answer different questions (Table V): the 54.0% over all declarations falls to 33.7% on finished answers and to 22.2% on finished answers whose manifests all parse, so the headline 64.0% of answers is not a count of exploitable builds. Repeats do not inflate the rates: the extractor counts each name once per answer, and repetition loops produce distinct names. By ecosystem the quality effect is uneven (Table V, bottom): ESP-IDF, highest overall at 95.2%, is 81.9% (77/94) on well-formed complete answers, the rest coming from manifests the component manager cannot read. Arduino, MicroPython and npm packages were declared only in install commands, so that subset holds none of them and its rate covers the other five ecosystems.

*Cut answers.* 562 answers (22.5%) reached the budget; of the 469 with a final text, 312 ended in a repetition loop (their last 2,000 characters compress below 30% of their size). Without the cut answers, 60.2% (CI [54.1, 65.9]) of the answers with a registry declaration name a phantom. Generated again with 4096/7680 tokens (the larger for the reasoning models), 252 cut answers finished and 310 reached the larger budget; with the extended answers in their place the rates are 61.1% (CI [50.3, 70.6]) per declaration and 63.9% per answer, and 34.3% and 61.1% over the answers that finish under either budget. Greedy decoding did not repeat itself exactly: 396 of 460 extended answers start with the cut one (the rest part early, and those of gpt-oss 20B, whose chat template dates the prompt, never do); on those, the declaration rate moves from 80.0% to 86.1%. Under one budget for both families, manifests written together with source code (F1) had 49.4% phantom declarations (2121/4291, CI [41.8, 57.3]) and manifest-only answers (F2) 56.9% (3928/6901, CI [49.5, 63.7]); per answer, 62.2% and 65.8%. Paired by model and task (898 pairs with registry declarations in both families), a phantom appeared only in the F1 answer in 65 pairs and only in the F2 answer in 85 (exact McNemar test, $p = 0.12$).

### B. RQ2: exposure

The responses contained 4484 distinct phantom names, of which 218 came from at least two models. By subtype: case or separator variants of a registered name, 67; PlatformIO names that exist under a different owner than the one declared, 57; registered PyPI projects without files, 3; crates with every version yanked, 0; names that match no registered name under these checks, 4357. By who could register them: any account holder, 2177; only the vendor (Espressif's own ESP namespace, and Arduino names that begin with "Arduino" in any case [12], [46]), 634; only the owner of another namespace (PlatformIO owners, npm scopes, other ESP namespaces), 305; only through a reviewed pull request, 923; only the owner of a registered name without installable files, 3; nobody, under the registries' rules on characters, length and variants of existing names (npm [47], [48], crates.io [49], PyPI, PlatformIO, ESP-IDF, Arduino [12], [46]), 418; untested, for variants in other registries, 24. Of 1483 PlatformIO registry declarations, 961 omitted the owner, the form the PlatformIO documentation advises against because it is ambiguous [13]. The artifact lists the most widely produced names and who could register them.

87 resolved declarations, with 27 distinct names, were post-horizon matches: the name the model wrote now resolves to a record created after the model's horizon. Table VI audits the names with the record and project dates of Section III. For 3 the repository behind the record has commits from before the horizon, so the model may have known the project, though not this record; for 8 the repository history itself starts after the horizon, so the name was most likely filled later by a project the model could not have known; for 1 the repository history starts after the record and cannot date the project; 3 are rosdep keys, a record that maps operating-system packages; 12 records name no repository. With repository creation dates as horizons there are 89 matches, with horizons 90 days later 77 and with horizons a year earlier 119.

The persistence follow-up re-asked 1286 phantom-producing (model, prompt) pairs at temperature 0.7; 1277 of them still have a phantom after the last fix of the extraction (6385 samples), and 14 pairs that gained one through later fixes were not re-asked. At least one phantom of the greedy answer reappeared in 2123 samples (33.2%, CI [31.5, 34.9]); without cut answers, greedy or resampled, in 34.3% (CI [32.4, 36.5]), and by model from 24.7% (Qwen2.5-Coder 1.5B) to 44.4% (Qwen2.5-Coder 14B). Of 5962 (pair, name) combinations, 69 reappeared in every sample and 4242 in none. Phantoms that reappear under sampling are the ones an adversary can harvest by querying the same model.

### C. RQ3: gating

Of 2082 manifests, 1291 are unsafe, 54 post-horizon and 737 clean. Table VII gives the decisions. In the observed world every policy except P0 rejects every unsafe manifest: a phantom refutes res, which the gate vetoes and which caps the upper index of I at $1 - \alpha_{\mathsf{res}} < \theta$. Of the 54 post-horizon manifests, P1 admits 54 and P2 54, while P3, $\mathrm{P3}_{\perp}$ and SCII reject 54, 54 and 54. On the 737 clean manifests, P1 admits 737, P3 711 (rejecting 26) and $\mathrm{P3}_{\perp}$ 644 (requesting evidence for 93); SCII admits 93, sends 547 to review and 97 to evidence requests. Requests arise where a mandatory item cannot be decided; here the causes were a URL or path that no registry resolves (67 manifests), a package whose only date, an upper bound, falls after the horizon (14 manifests), an owner-less PlatformIO name whose same-name packages were published before and after the horizon (12 manifests) and a package published in the 30 days before the horizon (4

TABLE IV

PHANTOM REGISTRY DECLARATIONS BY MODEL AND ECOSYSTEM (PHANTOMS/REGISTRY DECLARATIONS). PER MODEL: THE RATE OVER ALL ECOSYSTEMS AND THE SHARE OF RESPONSES WITH A PHANTOM AMONG RESPONSES WITH A REGISTRY DECLARATION. PER ECOSYSTEM: THE RATE WITH ITS 95% TASK-CLUSTER BOOTSTRAP INTERVAL (NONE FROM A SINGLE TASK) AND THE SHARE OF RESPONSES WITH A PHANTOM.

| Model | Arduino LM | PlatformIO | ESP-IDF | ROS 2 | crates.io | MicroPython | PyPI | npm | All | Responses |
|---|---|---|---|---|---|---|---|---|---|---|
| Qwen2.5-C 1.5B | 60/91 | 249/281 | 176/177 | 152/174 | 188/222 | 13/16 | 15/62 | 37/59 | 82.3% | 76.0% |
| Llama-3.2 3B | 775/801 | 50/58 | – | 183/251 | 342/407 | 46/52 | 45/103 | 42/57 | 85.8% | 90.2% |
| Qwen2.5-C 7B | 20/52 | 91/114 | 38/39 | 105/247 | 151/303 | 30/32 | 4/51 | 14/40 | 51.6% | 60.7% |
| DS-Coder 6.7B | 23/51 | 26/52 | 9/12 | 15/150 | 234/277 | 16/30 | 9/68 | 26/66 | 50.7% | 56.4% |
| Llama-3.1 8B | 14/36 | 318/356 | 5/5 | 190/384 | 217/283 | 31/45 | 50/100 | 21/46 | 67.4% | 79.3% |
| Qwen3 8B | 20/46 | 19/45 | 42/49 | 48/154 | 18/120 | 2/4 | 2/31 | 16/40 | 34.2% | 61.5% |
| Granite-3.3 8B | 24/38 | 25/36 | 3/3 | 54/173 | 11/82 | 16/22 | 23/68 | 40/55 | 41.1% | 72.7% |
| Gemma-3 12B | 25/52 | 94/137 | 105/107 | 108/330 | 24/159 | 23/37 | 8/78 | 38/75 | 43.6% | 71.7% |
| Qwen2.5-C 14B | 21/50 | 117/165 | 2/6 | 38/188 | 6/189 | 111/117 | 0/50 | 11/37 | 38.2% | 45.3% |
| Phi-4 14B | 18/53 | 22/58 | 10/13 | 27/239 | 53/243 | 1/1 | 52/129 | 10/39 | 24.9% | 48.3% |
| DS-Coder-V2 16B | 49/77 | 28/45 | 183/186 | 13/178 | 12/134 | 29/31 | 9/81 | 33/71 | 44.3% | 64.8% |
| gpt-oss 20B | 7/36 | 9/31 | 25/27 | 10/90 | 3/40 | 23/31 | 4/30 | 18/41 | 30.4% | 58.5% |
| Qwen2.5-C 32B | 42/74 | 42/105 | 129/140 | 10/191 | 18/258 | 21/37 | 3/53 | 12/37 | 30.9% | 50.8% |
| All | 1098/1457 | 1090/1483 | 727/764 | 953/2749 | 1277/2717 | 362/455 | 224/904 | 318/663 | 54.0% | 64.0% |
| Rate | 75.4% | 73.5% | 95.2% | 34.7% | 47.0% | 79.6% | 24.8% | 48.0% | | |
| 95% CI | [47, 87] | [60, 84] | [92, 97] | [27, 43] | [35, 57] | [72, 87] | [12, 40] | [37, 60] | | |
| Responses | 57% | 80% | 93% | 73% | 58% | 83% | 31% | 65% | | |

TABLE V

DENOMINATORS AND PHANTOM RATES BY SUBSET. TOP: THE RESPONSES AND THE RATES OF OTHER SUBSETS; THE RESPONSE-LEVEL RATE COUNTS ANSWERS, NOT BUILDS. BOTTOM: PHANTOMS/REGISTRY DECLARATIONS PER ECOSYSTEM, OVER ALL ANSWERS AND OVER FINISHED ANSWERS WHOSE MANIFESTS ALL PARSE (FOR ESP-IDF, MOST OF THE GAP IS MANIFESTS THE COMPONENT MANAGER CANNOT READ).

| | |
|---|---|
| Greedy responses | 2496 |
| with a declaration of any kind | 2083 |
| with a manifest (non-local, non-special) | 2082 |
| with a registry declaration | 2018 |
| Phantom rate, finished answers | 33.7% (6242 decl.) |
| Responses with $\geq 1$ phantom | 64.0% (1291/2018) |
| finished answers only | 60.2% (993/1649) |

| Ecosystem | All declarations | Well-formed, complete |
|---|---|---|
| Arduino LM | 75.4% (1098/1457) | – |
| PlatformIO | 73.5% (1090/1483) | 48.1% (260/541) |
| ESP-IDF | 95.2% (727/764) | 81.9% (77/94) |
| ROS 2 | 34.7% (953/2749) | 16.8% (186/1104) |
| crates.io | 47.0% (1277/2717) | 15.5% (185/1193) |
| MicroPython | 79.6% (362/455) | – |
| PyPI | 24.8% (224/904) | 13.9% (96/691) |
| npm | 48.0% (318/663) | – |
| All | 54.0% (6049/11192) | 22.2% (804/3623) |

TABLE VI

POST-HORIZON MATCHES: EARLIEST HORIZON $h$ OF THE MODELS THAT WROTE THE NAME, CREATION OF THE RECORD IT NOW RESOLVES TO, AND EARLIEST COMMIT OF THE REPOSITORY THAT THE RECORD NAMES.

| Ecosystem | Declared name | $h$ | Record | Repository |
|---|---|---|---|---|
| Arduino LM | `A4988` | 2023-10-29 | 2025-07-02 | 2025-07-02 |
| Arduino LM | `LiquidCrystal_I2C` | 2023-10-29 | 2025-07-13 | 2015-08-20 |
| Arduino LM | `ModbusRTU` | 2024-11-10 | 2025-05-22 | 2025-05-22 |
| crates.io | `can-bus` | 2024-09-18 | 2025-05-30 | none listed |
| crates.io | `embedded-hal-sdmmc` | 2024-09-18 | 2025-06-04 | none listed |
| crates.io | `flash` | 2025-04-28 | 2025-05-21 | none listed |
| ESP-IDF | `BME280` | 2023-10-29 | 2024-11-07 | none listed |
| ESP-IDF | `cJSON` | 2023-10-29 | 2025-10-06 | none listed |
| ESP-IDF | `mqtt` | 2024-11-06 | 2025-09-25 | none listed |
| npm | `node-red-contrib-hmac` | 2024-09-18 | 2026-06-26 | 2026-06-26 |
| PlatformIO | `LittleFS` | 2024-07-18 | 2025-01-30 | none listed |
| PlatformIO | `MAVLink` | 2023-10-29 | 2024-01-04 | none listed |
| ROS 2 | `px4_msgs` | 2024-06-14 | 2025-12-06 | 2019-01-21 |
| ROS 2 | `python3-regex` | 2025-03-12 | 2025-04-29 | rosdep key |
| ROS 2 | `python3-wheel` | 2024-09-18 | 2025-02-19 | rosdep key |
| … and 12 further names (artifact) | | | | |

manifests). With a gate-only profile ($\alpha_{\mathsf{pin}} = \alpha_{\mathsf{prov}} = 0$) SCII admits 640 clean manifests, requests evidence for 97 and still rejects every unsafe one; with $\delta = 0$ it decides every manifest exactly as $\mathrm{P3}_{\perp}$ (checked on all manifests and worlds).

*Where a date filter stops.* The indices of the Arduino, ROS and MicroPython installers carry no dates. With them alone, P3 rejects 217 clean manifests (191 only for want of a date) and SCII sends 188 more to evidence requests. Of the clean and post-horizon manifests with a dependency dated only by registry histories, P3 stops 227, which SCII sends to requests; with the history dates, 36 are rejected as post-horizon matches, $\mathrm{P3}_{\perp}$ admits 190 and SCII sends 188 to review, leaving 3 open (reading the histories took 19 s). The request, not the index arithmetic, turns a missing date into evidence.

*Availability.* Reviews come from the graded index, not from the gate: without provenance a resolved registry dependency contributes at most $1 - \alpha_{\mathsf{prov}} = 0.9$, so the product kernel admits a change with $k$ such dependencies only if $(1-\alpha_{\mathsf{prov}})^k \geq \theta$, that is, for $k \leq 1$ with the assigned profile. Of the clean manifests, 89 of 178 with one registry dependency were admitted and 0 of 216 with two. The threshold trades availability for review load without touching safety: 13% of the clean manifests are admitted with the assigned profile, 42% with $\theta = 0.7$, 45% with $\theta = 0.5$, and 37% with $\alpha_{\mathsf{prov}} = 0$; blocking never changes. Much of this friction follows from where pins exist. On the 286 clean manifests whose crates.io, npm and PyPI declarations the tools resolved and locked, SCII auto-admits 89 (31.1%), sends 184 to review

and requests evidence for 13, against 13% over all clean manifests: elsewhere no lock file pins a digest, an unpinned registry dependency scores at most $\alpha_{\mathsf{res}} + \alpha_{\mathsf{est}} < \theta$, and SCII admits no manifest with a registry dependency there. Of the 644 clean manifests SCII does not admit, the lock check covers 341: 103 would not install as written (versions that do not exist, unresolvable dependencies, conflicts) and 238 are stopped by the rule alone; for 303 without a lock check it cannot tell. A review concerns a median of 2 registry declarations (at most 17). With the capability alone instead of the written lock files, SCII would admit 112 clean manifests, and 133 if it verified the attestations npm and PyPI already serve.

*Squat world.* The resolves-now check admits 99.7% of the unsafe manifests from day one and the age rule 99.7% from day 30. If only the phantoms open to any account holder are squatted, resolves-now admits 590 of the 590 unsafe manifests whose phantoms are all open, 46% of all 1291. The optimistic variant O admits 18% at every lag (squats in registries dated from above only); P3, P3$_\perp$, T and SCII admit none at any lag. Meeting the squats for the first time, SCII rejects 1063 unsafe manifests and requests evidence for 228, whose squats only have an upper bound; with the absence record of the snapshot it rejects 1291. With the assigned profile the index-only rule admits 0%, because $\alpha_{\mathsf{res}} + \alpha_{\mathsf{pin}} = 0.6 < \theta$. Over 1000 random profiles, the condition of Section IV for a pinned squat holds in 19.9% of the profiles; the index-only rule admitted at least one squatted manifest under 19.9% of all profiles, exactly those that satisfy the condition, and SCII under none.

### D. Replay on real installers

We registered 9 open phantom names of the pilot's answers (three each in PyPI, npm and crates.io) as harmless packages that write a marker file when their build or install code runs, in local registries (a PEP 691/700 index, Verdaccio 6.10.7 without uplinks, a sparse Cargo registry), with one control package per registry dated 180 days before the horizon; the replay ran in a network namespace whose only interface is loopback. With the defaults of pip 24.0, uv 0.11.32, npm 10.9.4 and Cargo 1.97.0 (P1), 9 of the 9 squats ran their code; the date filters of uv (`--exclude-newer`) and npm (`--before`) at the horizon refused 6 of 6 squat installations, and pip and Cargo have none. The gate rejected 6 squats on first contact and asked for evidence for the 3 on PyPI, dated from above only, and rejected 9 with the absence record of an earlier lookup: no squat reached an installer, and the controls were admitted, locked without running code and installed (3 of 3). Served from an index without upload times, as a find-links page or an older mirror serves it, a control was refused by uv's date filter, while the gate asked for evidence, read the record date from the origin registry (1 request, 22 ms) and admitted it. Locking a squat that ships only a source distribution with uv without `--no-build` ran its build backend (3 of 3), so the gate must decide before any lock step.

*ESP-IDF stand-in.* The pilot's ESP-IDF phantoms lie in Espressif's namespace, closed to outsiders, so we replayed a stand-in, not a squat: a harmless component under one phantom's pseudonym in a placeholder namespace of a mock ESP Component Registry, declared by a minimal application; it shows what the build does once such a name resolves. The component manager 3.1.2, which has no date filter, downloaded it without running its code (code ran in 0 of 1); `idf.py build` of ESP-IDF 6.0.3 (linux host target) then ran its CMake file and compiled it (1 of 1). The gate rejected it before any download (1 of 1), and the control was built from the lock the gate measured (1 of 1).

### E. RQ4: evidence and cost

Establishment could be decided for 98.4% of resolved declarations; Table II shows the split by ecosystem. Where the registry bounds a record from above only, as PyPI does, a manifest without a refuted item can end in REQUEST: the gate states which evidence is missing rather than guessing. *Lock files.* For the 788 pairs of an answer and an ecosystem with resolved crates.io, npm or PyPI declarations, the tools themselves (Cargo 1.97.0, npm 10.9.4, uv 0.11.32) wrote a lock file in resolve-only mode for 522 (CPython 3.13, no source builds). The capability does not imply a lock: 280 declarations name versions that do not exist, and only 1205 of 2465 declarations (48.9%) end up pinned. The registries serve provenance attestations for 35/343 locked npm and 86/308 PyPI declarations (presence only). A lock took 0.35 s per manifest at the median (at most 27 s). A cold registry lookup took 343 ms at the median (95th percentile 741 ms; 156 lookups, 152 for ESP-IDF names), evaluating one policy on one manifest about 16 $\mu$s.

## VII. Discussion

*Anchor to the producer, not to the clock.* A resolves-now check asks a question that the attacker answers; an age rule only delays the answer. Whether the record existed when the producer's weights did never changes, and the audit shows why the record, not the project, is the unit: an old project can sit behind a new record.

*What the index adds to a date filter.* Where record dates exist, a record-age filter at the horizon blocks squats as well as the rule. The case for the rule is therefore narrow: the horizon comes from the producer, which no installer knows; dates come from registry histories that installers do not read; and an undecidable date becomes a request that names the missing evidence instead of a rejection (Section VI-D). Reading absent evidence as refuted (T) rejects clean manifests; reading it as confirmed (O) admits squats where records are dated from above only. Blocking is the gate's job and needs none of the weights; the graded index is an optional layer on top that lets a project demand pins and provenance before a change goes in without a human, in the same decision record. Its auto-admission pays off where lock files pin digests (31.1% of the clean manifests that lock) and registries serve provenance; elsewhere a project can run the gate alone and send clean manifests to review.

TABLE VII

GATE DECISIONS. OBSERVED WORLD (PINS FROM THE LOCK FILES THE TOOLS WROTE): A = ADMIT, ? = REVIEW OR REQUEST, X = REJECT, FOR UNSAFE ($U$), POST-HORIZON ($H$) AND CLEAN ($C$) MANIFESTS; FOR CLEAN MANIFESTS R = REVIEW AND Q = REQUEST. SQUAT WORLD: UNSAFE MANIFESTS ADMITTED WHEN THE GATE RUNS $L$ DAYS AFTER EVERY PHANTOM WAS SQUATTED, AND ON THE FIRST DAY WHEN ONLY OPEN NAMES WERE SQUATTED.

| | Observed world | | | | | | | | | | Squat world, $U$ admitted | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Unsafe $U$ | | | Post-horizon $H$ | | | Clean $C$ | | | | $L$=0 | $L$=30 | $L$=365 | open, $L$=0 |
| Policy | A | ? | X | A | ? | X | A | R | Q | X | | | | |
| P0 accept all | 1291 | 0 | 0 | 54 | 0 | 0 | 737 | 0 | 0 | 0 | 1291 | 1291 | 1291 | 1291 |
| P1 resolves now | 0 | 0 | 1291 | 54 | 0 | 0 | 737 | 0 | 0 | 0 | 1287 | 1287 | 1287 | 590 |
| P2 age $\geq$30 d | 0 | 0 | 1291 | 54 | 0 | 0 | 737 | 0 | 0 | 0 | 0 | 1287 | 1287 | 0 |
| P3 as of $h$ | 0 | 0 | 1291 | 0 | 0 | 54 | 711 | 0 | 0 | 26 | 0 | 0 | 0 | 0 |
| P3$_\perp$ as of $h$, unknown asks | 0 | 0 | 1291 | 0 | 0 | 54 | 644 | 0 | 93 | 0 | 0 | 0 | 0 | 0 |
| I index only | 0 | 0 | 1291 | 0 | 0 | 54 | 93 | 644 | 0 | 0 | 0 | 0 | 0 | 0 |
| T two-state | 0 | 0 | 1291 | 0 | 0 | 54 | 93 | 547 | 0 | 97 | 0 | 0 | 0 | 0 |
| O optimistic | 0 | 0 | 1291 | 0 | 0 | 54 | 737 | 0 | 0 | 0 | 228 | 228 | 228 | 92 |
| SCII, rule (3) | 0 | 0 | 1291 | 0 | 0 | 54 | 93 | 547 | 97 | 0 | 0 | 0 | 0 | 0 |

*Calibrating friction.* A project that wants changes with up to $K$ dependencies without provenance admitted unreviewed must choose $\theta \leq (1 - \alpha_{\mathsf{prov}})^K$, or set $\alpha_{\mathsf{prov}} = 0$ until its registries offer provenance; Propositions 1–3 hold for all weights and thresholds while res and est stay mandatory. Most requests come from URLs and paths (67 of 97), which need evidence no registry gives, such as a pinned commit's date.

*Place in a CPS pipeline.* The gate runs before signing, so only admitted changes reach the images TUF or Uptane sign [27], [28]; its items can be emitted as in-toto attestations [29] and serve as evidence for the dependency control IEC 62443-4-1 and SSDF require [33], [34]. It must also run before any step that can execute package code (for ESP-IDF the build itself; Section VI-D). Code generators should record the model, its weight hash and its horizon with every change; for a person the writing time is a natural horizon.

## VIII. THREATS TO VALIDITY

*Construct.* Extraction ignores prose and may miss or misparse declarations; its rules are unit-tested, the official parsers agree with it on well-formed files, and the manual validation, one joint annotation without a measure of agreement, found gaps that this version keeps (tables of an unparsed `Cargo.toml`, requirement lists without a fence). Phantom labels follow lookups that model the installer's name resolution, checked for Cargo (case and separator variants) and npm (capital letters); for PlatformIO and ESP we follow their APIs, and `arduino-cli` adjusts a name's case when exactly one library matches it up to case. The horizon bounds the training cut-off from above; moving every horizon 180 days earlier gives 103 post-horizon matches instead of 87 and establishment decidable for 98.0% of resolved declarations (98.4% with the reported horizons). Dates from registry git histories are those of the main line, which the registry sets. A post-horizon match may be a legitimate late registration of an older project, as the audit shows; the rule rejects it all the same, because the record the installer would fetch is new. The squat worlds ignore what a registry would do about a malicious package once reported.

*Internal.* Registry state changes daily; all answers carry retrieval times. A registry that backdates records or lets a deleted name be re-registered with its old history breaks the assumption of Proposition 2; such cases call for name-history evidence in prov. Quantisation, server defaults and chat templates can change outputs; the runner records them. Greedy answers of the inference server were not reproducible byte for byte (Section VI-A); the artifact keeps the answers as generated, and every number is computed from them.

*External.* The models are open-weight models up to 32.8 billion parameters, quantised to about four bits and run with one library's defaults and greedy decoding; commercial models, other decoding settings and agents with retrieval may behave differently. The 96 tasks are hand-written; other registries, tasks and retrieval-augmented agents may differ.

## IX. CONCLUSION

In our study, LLM-written build manifests for CPS named packages that do not exist, and the common defense admits them once an adversary registers the name. Anchoring establishment to the date at which the producer's weights existed, and gating on three-state evidence with a non-compensable rule, closes this gap for names registered after that horizon, for any weights and gating time, under valid record dates. Its value over a plain date filter lies in the producer's horizon, dates recovered from registry histories, and requests instead of rejections where dates are missing or one-sided. Next come an independent annotation of extraction and classification, retrieval agents with logged horizons, and transitive coverage through the lock file.

## DATA AVAILABILITY

The anonymized artifact (*[anonymous link added at submission]*) holds the prompts, run configurations with model hashes and hub commits, the harness and tests, registry snapshots, pseudonymised declarations and answers, the lock-file, audit and replay records, and the commands and provenance behind every number. Raw generations and cached registry answers follow, with a DOI, once the registries are informed.

## Acknowledgment

In line with IEEE policy on AI-generated content, the authors disclose that an AI assistant (Claude, Anthropic) was used to draft text, code and figures under the authors' direction; the authors reviewed and take responsibility for all content.

## References

[1] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of attacks on open-source software supply chains," in *Proc. IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 1509–1526.

[2] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, ser. LNCS, 2020, pp. 23–43.

[3] S. Cordey, "Software supply chain attacks: An illustrated typological review," CSS Cyberdefense Reports, Center for Security Studies, ETH Zürich, 2023.

[4] P. Przymus and T. Durieux, "Wolves in the repository: A software engineering analysis of the XZ Utils supply chain attack," in *Proc. IEEE/ACM 22nd Int. Conf. on Mining Software Repositories (MSR)*, 2025, pp. 91–102.

[5] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, and M. Jadliwala, "We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs," in *Proc. 34th USENIX Security Symposium*, 2025, pp. 3687–3706.

[6] A. Churilov, "The range shrinks, the threat remains: Re-evaluating LLM package hallucinations on the 2026 frontier-model cohort," arXiv:2605.17062, 2026.

[7] B. Lanyado, "Diving into AI package hallucinations," Lasso Security research blog, https://www.lasso.security/blog/ai-package-hallucinations, Mar. 2024.

[8] S. Neupane, G. Holmes, E. Wyss, D. Davidson, and L. De Carli, "Beyond typosquatting: An in-depth look at package confusion," in *Proc. 32nd USENIX Security Symposium*, 2023, pp. 3439–3456.

[9] A. Birsan, "Dependency confusion: How I hacked into Apple, Microsoft and dozens of other companies," Blog post, https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610, Feb. 2021.

[10] A. Krishna, E. Galinkin, L. Derczynski, and J. Martin, "Importing phantoms: Measuring LLM package hallucination vulnerabilities," arXiv:2501.19012, 2025.

[11] J. Zheng, H. Guan, and Y. Liu, "When LLMs invent Rust crates: An empirical study of hallucination patterns and mitigation," arXiv:2606.08444, 2026.

[12] Arduino, "Arduino Library Manager Registry: Submission Process," https://github.com/arduino/library-registry, 2026.

[13] PlatformIO, "PlatformIO Library Manager: Dependency Management," https://docs.platformio.org/en/latest/librarymanager/dependencies.html, 2026.

[14] Espressif Systems, "IDF Component Manager: Manifest File Reference," https://docs.espressif.com/projects/idf-component-manager/, 2026.

[15] Open Robotics, "rosdistro: Review Guidelines," https://github.com/ros/rosdistro/blob/master/REVIEW_GUIDELINES.md, 2026.

[16] J. Cheng, M. Marone, O. Weller, D. Lawrie, D. Khashabi, and B. Van Durme, "Dated data: Tracing knowledge cutoffs in large language models," arXiv:2403.12958, 2024.

[17] npm, Inc., "npm docs: options `before` and `min-release-age`," https://docs.npmjs.com/cli/v11/using-npm/config, 2026.

[18] Astral, "uv docs: resolution, `--exclude-newer`," https://docs.astral.sh/uv/concepts/resolution/, 2026.

[19] Anonymous, "Under submission. Details omitted for double-blind reviewing," 2026.

[20] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *Proc. 28th USENIX Security Symposium*, 2019, pp. 995–1010.

[21] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, "What are weak links in the npm supply chain?" in *Proc. 44th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2022, pp. 331–340.

[22] Y. Shen, X. Gao, H. Sun, and Y. Guo, "Understanding vulnerabilities in software supply chains," *Empirical Software Engineering*, vol. 30, no. 1, 2025, art. no. 20.

[23] L. Williams *et al.*, "Research directions in software supply chain security," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–38, 2025.

[24] S. Hamer, J. Bowen, M. N. Haque, R. Hines, C. Madden, and L. Williams, "Closing the chain: How to reduce your risk of being SolarWinds, Log4j, or XZ Utils," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 1841–1853.

[25] Y. Zhao, M. Wu, X. Hu, and X. Xia, "HFUZZER: Testing large language models for package hallucinations via phrase-based fuzzing," in *Proc. 40th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2025.

[26] X. Liu, Y. Liu, Y. Zhang, J. Li, and S.-M. Hu, "PackMonitor: Enabling zero package hallucinations through decoding-time monitoring," arXiv:2602.20717, 2026.

[27] J. Samuel, N. Mathewson, J. Cappos, and R. Dingledine, "Survivable key compromise in software update systems," in *Proc. 17th ACM Conf. on Computer and Communications Security (CCS)*, 2010, pp. 61–72.

[28] T. K. Kuppusamy *et al.*, "Uptane: Securing software updates for automobiles," in *Proc. 14th escar Europe*, 2016.

[29] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proc. 28th USENIX Security Symposium*, 2019, pp. 1393–1410.

[30] OpenSSF, "SLSA specification v1.1," https://slsa.dev/spec/v1.1/, Apr. 2025.

[31] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software signing for everybody," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2022, pp. 2353–2367.

[32] J. Boyens *et al.*, "Cybersecurity supply chain risk management practices for systems and organizations," National Institute of Standards and Technology, NIST Special Publication 800-161r1-upd1, Nov. 2024.

[33] M. Souppaya, K. Scarfone, and D. Dodson, "Secure software development framework (SSDF) version 1.1," National Institute of Standards and Technology, NIST Special Publication 800-218, 2022.

[34] International Electrotechnical Commission, "IEC 62443-4-1:2018 Security for Industrial Automation and Control Systems – Part 4-1: Secure Product Development Lifecycle Requirements," International Standard, 2018.

[35] European Parliament and Council of the European Union, "Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)," Official Journal of the European Union, Nov. 2024.

[36] B. Hui *et al.*, "Qwen2.5-Coder technical report," arXiv:2409.12186, 2024.

[37] A. Grattafiori *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[38] D. Guo *et al.*, "DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence," arXiv:2401.14196, 2024.

[39] DeepSeek-AI *et al.*, "DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence," arXiv:2406.11931, 2024.

[40] IBM Granite Team, "Granite-3.3-8B-Instruct model card," https://huggingface.co/ibm-granite/granite-3.3-8b-instruct, 2025.

[41] Gemma Team, "Gemma 3 technical report," arXiv:2503.19786, 2025.

[42] M. Abdin *et al.*, "Phi-4 technical report," arXiv:2412.08905, 2024.

[43] A. Yang *et al.*, "Qwen3 technical report," arXiv:2505.09388, 2025.

[44] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," arXiv:2508.10925, 2025.

[45] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 69, no. 3, pp. 369–390, 2007.

[46] Arduino, "Arduino Lint: Library rules," https://arduino.github.io/arduino-lint/1.3/rules/library, 2026.

[47] npm, Inc., "npm docs: the `name` field of `package.json`," https://docs.npmjs.com/cli/v11/configuring-npm/package-json, 2026.

[48] ——, "New package moniker rules," npm blog, https://blog.npmjs.org/post/168978377570/new-package-moniker-rules, Dec. 2017.

[49] crates.io, "Source code: canonical-name index and crate-name validation," https://github.com/rust-lang/crates.io, commit 554bc83, 2026.